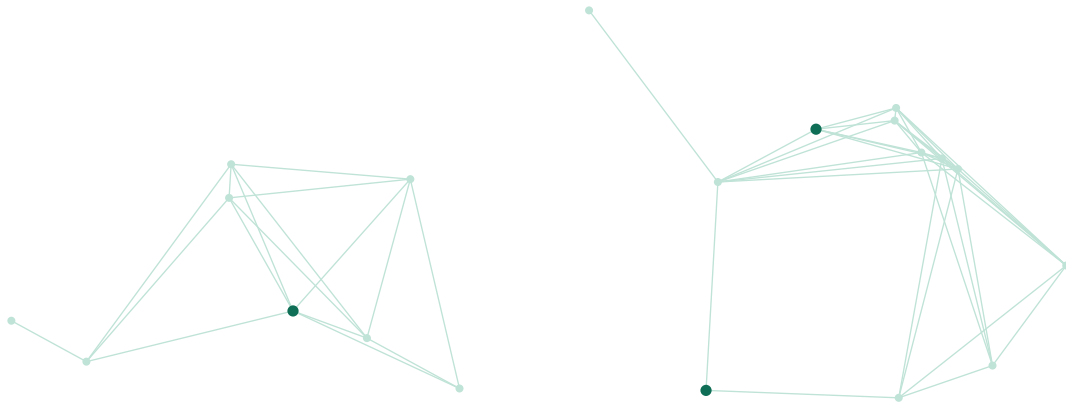




EIN LERN-EBOOK ZUM SELBST GEBAUTEN PROJEKT

Mini-Vektordatenbank mit RBAC

*Vektoren, Embeddings und serverseitige Zugriffskontrolle von Grund auf verstehen — inklusive
JWT-Login, Persistenz und Interview-Vorbereitung*

`github.com/Arian710/mini-vector-rbac` • Oktober 2026

ORIENTIERUNG

Inhaltsverzeichnis

Inhaltsverzeichnis	2
Einleitung	3
1. Was ist ein Vektor?	5
2. Was ist ein Embedding?	5
3. Cosine Similarity	5
4. Brute-Force-Suche	6
5. Warum RBAC serverseitig passieren muss	6
<i>Client vs. Server</i>	7
6. Architektur-Überblick	9
7. Wichtige Software-Design-Konzepte	10
<i>Abstraktion / Interface</i>	10
<i>Dependency Injection</i>	10
<i>Allow-Listing statt Deny-Listing</i>	10
8. Die Web-API: REST, JSON, FastAPI	10
9. Automatisierte Tests	11
10. Passwort-Hashing	13
11. JWT-Login	13
12. SQLite-Persistenz	14
13. Authentication vs. Authorization	15
14. Audit-Log	16
15. Multi-Tenancy	16
16. Die konkreten Schritte, in Reihenfolge	19
17. Interview-Fragenkatalog	21
18. Glossar	23

START

Einleitung

Dieses eBook fasst alles zusammen, was wir beim Bau einer kleinen, aber vollständigen Vektordatenbank mit rollenbasierter Zugriffskontrolle (RBAC) gemeinsam durchgearbeitet haben. Es ist bewusst für jemanden geschrieben, der keinen tiefen IT-Hintergrund hat und länger nicht mehr programmiert hat — jedes Konzept wird zuerst in einfacher Sprache mit einem kleinen Beispiel erklärt, bevor der zugehörige Code gezeigt wird.

Der rote Faden des ganzen Projekts ist eine einzige Kernidee: **Wenn eine Suche per Künstlicher Intelligenz über Firmendokumente läuft, muss die Zugriffsprüfung zwingend auf dem Server passieren — nie nur in der Oberfläche.** Alles, was wir gebaut haben, dient letztlich dazu, dieses eine Prinzip technisch wasserdicht umzusetzen: von der Mathematik hinter der Suche bis zum echten Login mit Token.

Das Buch ist in fünf Teile gegliedert: zuerst die mathematischen Grundkonzepte, dann die Architektur des Projekts, danach die Produktiv-Features (Login, Persistenz, Audit-Log, Multi-Tenancy), eine Chronologie der tatsächlichen Bauschritte, und zum Schluss ein Referenzteil mit Interview-Fragen und einem Glossar zum Nachschlagen.

01

TEIL 1

Die Grundkonzepte

Vektor · Embedding · Cosine Similarity · Brute-Force · RBAC

KONZEPT · 01

1. Was ist ein Vektor?

Ein Vektor ist zunächst nichts Mystisches — er ist einfach eine **Liste von Zahlen, die eine Position im Raum beschreibt**. Stell dir eine Wohnung vor, die du mit zwei Zahlen beschreibst: Quadratmeter und Anzahl Zimmer.

python

```
wohnung_a = [80, 3] # 80 m², 3 Zimmer  
wohnung_b = [45, 1] # 45 m², 1 Zimmer
```

Das ist ein Vektor mit 2 Dimensionen. Du kannst dir jede Wohnung als Punkt in einem Koordinatensystem vorstellen: x-Achse = Quadratmeter, y-Achse = Zimmerzahl. Zwei Wohnungen, die sich ähnlich sind, liegen als Punkte nah beieinander — zwei sehr unterschiedliche Wohnungen liegen weit auseinander. **Räumliche Nähe entspricht inhaltlicher Ähnlichkeit** — das ist die Grundidee, auf der alles Weitere aufbaut.

Ein Vektor muss nicht bei 2 Zahlen bleiben. Du könntest eine dritte Zahl hinzunehmen (Stockwerk), eine vierte (Baujahr), eine zehnte... Wir können uns Räume mit mehr als 3 Dimensionen nicht mehr vorstellen oder zeichnen — aber die Mathematik (Abstand berechnen, Ähnlichkeit berechnen) funktioniert bei 3 Zahlen genauso wie bei 300 oder 1536. Das wird gleich wichtig, denn Embeddings sind genau solche langen Zahlenlisten.

KONZEPT · 02

2. Was ist ein Embedding?

Ein Embedding ist nichts anderes als der Vektor aus Kapitel 1 — nur mit einer besonderen Eigenschaft: Er beschreibt nicht eine Wohnung, sondern die **Bedeutung** von Text.

Wer entscheidet, welche Zahlen ein Satz bekommt? Bei der Wohnung war das einfach — Quadratmeter misst man mit einem Zollstock. Bei einem Satz wie „Der Kunde will sein Geld zurück“ gibt es kein Maßband für Bedeutung. Deshalb wird das nicht von Hand festgelegt, sondern von einem **trainierten neuronalen Netz** (dem Embedding-Modell) übernommen: Text rein, eine Liste von z.B. 1536 Zahlen raus.

TIPP

Zwei Sätze wie „Rechnung wurde nicht bezahlt“ und „Zahlung ist ausständig“ teilen kein einziges Wort, meinen aber fast dasselbe. Ein gutes Embedding-Modell erzeugt dafür Vektoren, die im Raum **nah beieinander** liegen — weil das Modell nicht auf einzelne Wörter reagiert, sondern worum es im Satz geht gelernt hat.

KONZEPT · 03

3. Cosine Similarity

Jetzt brauchen wir eine Zahl, die ausdrückt, wie ähnlich zwei Vektoren sind. Der naive erste Gedanke wäre, den reinen Abstand zu messen (Pythagoras). Das ist bei Embeddings aber das falsche Werkzeug.

Schritt für Schritt von Hand gerechnet

Zwei Vektoren: $A = [2, 1]$, $B = [4, 2]$. B ist exakt A, nur jede Zahl verdoppelt.

von hand

Skalarprodukt: $A \cdot B = (2 \times 4) + (1 \times 2) = 10$

Länge $|A|$: $\sqrt{2^2 + 1^2} = \sqrt{5} \approx 2.236$

Länge $|B|$: $\sqrt{4^2 + 2^2} = \sqrt{20} \approx 4.472$

$\cos(A, B) = (A \cdot B) / (|A| \times |B|) = 10 / (2.236 \times 4.472) = 1.0$

Ergebnis 1.0 bedeutet: A und B zeigen in exakt dieselbe **Richtung**, obwohl ihre reinen Zahlenwerte und ihr Abstand komplett unterschiedlich sind. Genau deshalb ist der Winkel das richtige Maß für Embeddings: Die Länge eines Embedding-Vektors kann technisch schwanken, ohne dass sich die Bedeutung ändert — Cosine Similarity ignoriert diese Länge bewusst und schaut nur auf die Richtung.

Zum Vergleich: $C = [-1, 2]$ steht im rechten Winkel zu A. Skalarprodukt $A \cdot C = -2 + 2 = 0$, also $\cos(A, C) = 0$ — keinerlei Ähnlichkeit.

Merkregel: 1.0 = gleiche Richtung/Bedeutung · 0 = unabhängig/unverwandt · -1.0 = genau entgegengesetzt.

So steht es im Code (db.py)

python

```
def cosine_similarity(a, b):
    norm_a = np.linalg.norm(a)
    norm_b = np.linalg.norm(b)
    if norm_a == 0 or norm_b == 0:
        return 0.0
    return float(np.dot(a, b) / (norm_a * norm_b))
```

KONZEPT · 04

4. Brute-Force-Suche

Die Idee ist stur-einfach: Eine Suchanfrage wird mit demselben Embedding-Modell in einen Vektor übersetzt. Dann wird **jeder einzelne** gespeicherte Vektor durchgerechnet, die Cosine Similarity gemerkt, am Ende absteigend sortiert und die Top-Treffer zurückgegeben. Kein Index, kein Trick — daher der Name „brute force“, roh mit Gewalt.

Bei 4 Tickets ist das eine Millisekunde Arbeit. Bei 10 Millionen Tickets bedeutet eine einzige Anfrage 10 Millionen Berechnungen — das skaliert linear mit der Datenmenge. Für unsere Größenordnung (ein paar Tausend Dokumente) ist das aber völlig ausreichend; ein Index wie HNSW lohnt sich erst bei sehr großen Datenmengen und wurde hier bewusst nicht gebaut.

KONZEPT · 05

5. Warum RBAC serverseitig passieren muss

RBAC (Role-Based Access Control) heißt: Zugriff wird nicht pro Person einzeln vergeben, sondern über Rollen. „Rolle management darf Finanz-Tickets sehen, Rolle support nicht.“

Client vs. Server

Client = Code, der auf dem Gerät des Nutzers läuft (Browser, App). Der Nutzer kann diesen Code öffnen, auslesen, umgehen — Browser-Entwicklertools (F12) sind serienmäßig eingebaut. **Server** = Code auf eurer eigenen Infrastruktur, den der Nutzer nie zu sehen bekommt.

Die Kernregel: Alles, was auf dem Client läuft, ist keine Sicherheitsgrenze — bestenfalls Komfort für die Oberfläche. Ein Filter im Client filtert nicht den Zugriff, sondern nur die Anzeige.

WICHTIG

Konkretes Leck-Beispiel: Ein sensibles HR-Ticket („Gehaltserhöhung genehmigt, 78.000€“) hat `allowed_roles=["management"]`. Würde der Server ALLE Treffer zurückschicken und erst der Client entscheiden, was angezeigt wird, wäre das Ticket zum Zeitpunkt der Anzeige-Entscheidung bereits im Klartext über das Netzwerk beim Nutzer angekommen. Ein Support-Mitarbeiter öffnet die Entwicklertools Netzwerk-Tab sieht die rohe Server-Antwort. Oder er ruft die API einfach direkt per curl auf, ganz ohne die offizielle Oberfläche.

Deshalb gehört der Filter zwingend in den Server-Code, als fester Bestandteil der Suche selbst — nicht als nachgelagerter Schritt beim Aufrufer.

02

TEIL 2

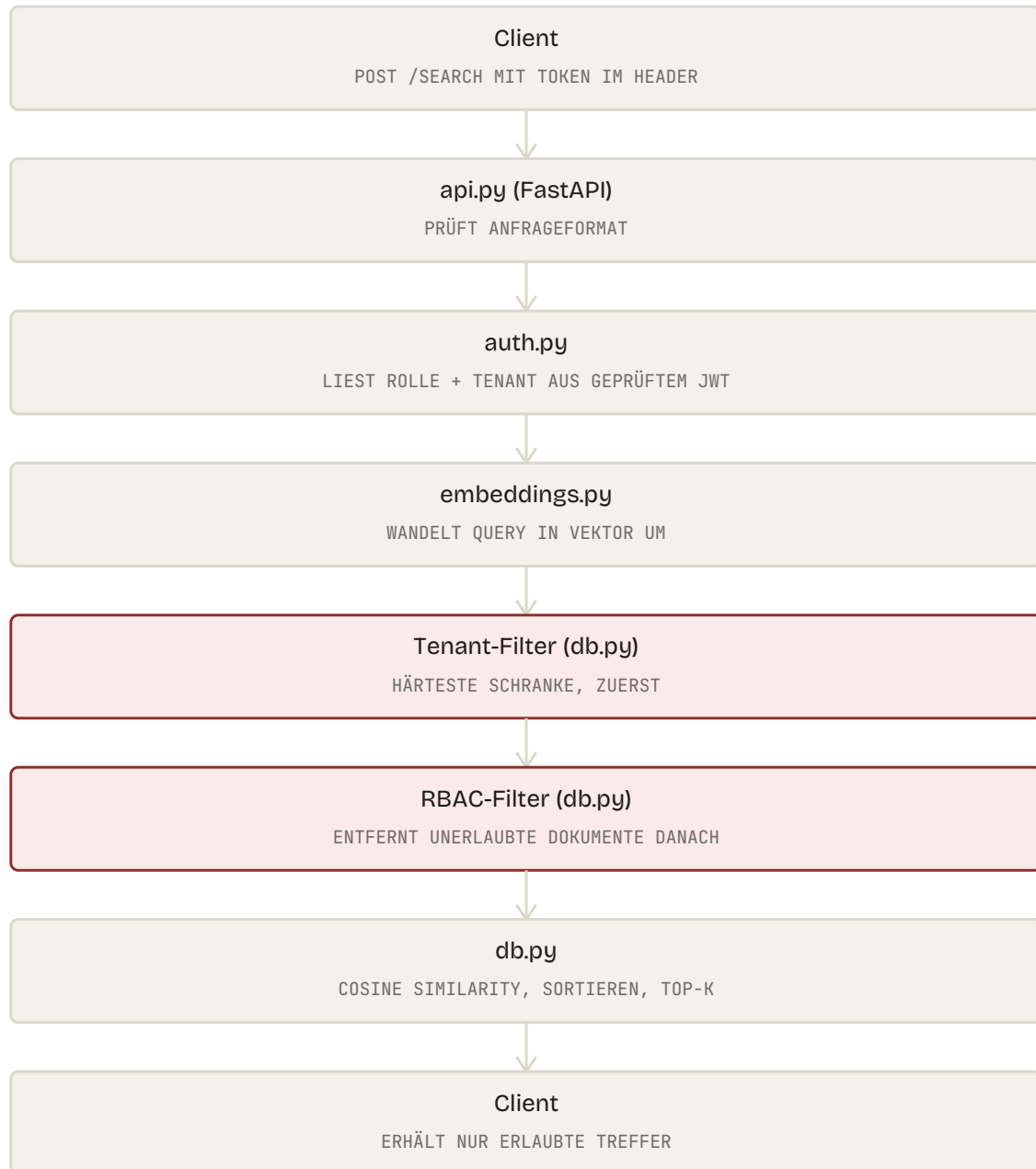
Die Architektur unseres Projekts

Dateien · Design-Patterns · Web-API · Tests

ARCHITEKTUR · 06

6. Architektur-Überblick

So läuft eine Suchanfrage heute durch das System — vom eintreffenden Request bis zur gefilterten Antwort:



Jede Datei hat genau eine Aufgabe (**Separation of Concerns**, Trennung der Zuständigkeiten):

- **data.py** — Beispiel-Tickets für zwei Mandanten, reine Daten, keine Logik
- **auth.py** — Login, Passwort-Prüfung, JWT-Erzeugung/-Prüfung (inkl. Tenant)
- **storage.py** — SQLite-Persistenz, trennt Speicherung von Suchlogik
- **embeddings.py** — Text-zu-Vektor-Übersetzung, austauschbar

- **db.py** — die eigentliche Suchmaschine: Cosine Similarity, Brute-Force, Tenant- und RBAC-Filter
- **api.py** — Web-Schicht (FastAPI), nimmt HTTP-Anfragen entgegen
- **seed_data.py** — befüllt die Datenbank mit Beispieldaten für beide Mandanten
- **test_rbac.py / test_auth.py / test_audit.py / test_tenancy.py** — automatisierte Tests

ARCHITEKTUR · 07

7. Wichtige Software-Design-Konzepte

Abstraktion / Interface

Eine Abstraktion ist ein „Vertrag“: Sie legt fest, *was* eine Komponente können muss, nicht *wie*. In `embeddings.py` gibt es eine Basis-Klasse `Embedder`, die nur verlangt: „jede Umsetzung braucht eine `embed(text)`-Methode, die einen Vektor zurückgibt.“

```
python
class Embedder(ABC):
    @abstractmethod
    def embed(self, text: str) -> np.ndarray:
        raise NotImplementedError
```

Zwei konkrete Umsetzungen halten sich an diesen Vertrag: `HashingEmbedder` (kostenloser, deterministischer Platzhalter) und `AzureOpenAIEmbedder` (echtes, trainiertes Modell über die Azure-API).

Dependency Injection

Statt dass `MiniVectorDB` sich selbst aussucht, wie sie Text in Vektoren umwandelt, wird ihr das von außen „gereicht“ (injiziert):

```
python
class MiniVectorDB:
    def __init__(self, embedder: Embedder = None):
        self._embedder = embedder or HashingEmbedder()
```

Dadurch kann `main.py` mit echten Azure-Embeddings laufen, während `test_rbac.py` bewusst den kostenlosen `HashingEmbedder` erzwingt — offline, deterministisch, ohne dass sich `db.py` selbst ändern müsste.

Allow-Listing statt Deny-Listing

Ein Dokument ist nur sichtbar, wenn seine Rolle **explizit erlaubt** ist (`allowed_roles`), nicht „sichtbar, außer gesperrt“. Sicherer, weil ein vergessener Eintrag im Zweifel zu weniger statt zu mehr Sichtbarkeit führt.

ARCHITEKTUR · 08

8. Die Web-API: REST, JSON, FastAPI

Eine **Web-API** ist ein Programm, das über das Netzwerk erreichbar ist. Ein Client schickt eine **Request** (Anfrage), bekommt eine **Response** (Antwort) zurück — meist im **JSON**-Format, einem einfachen Text-Format für strukturierte Daten. FastAPI ist das Python-Framework, das wir dafür benutzen.

Pydantic-Modelle prüfen automatisch, ob eine eingehende Anfrage das richtige Format hat, bevor eigener Code läuft — Eingabe-Validierung an der Netzwerkgrenze:

python

```
class SearchRequest(BaseModel):
    query: str
    top_k: int = 3
```

Der wichtigste Design-Punkt aus Kapitel 5 wird hier an der echten Netzwerkgrenze durchgesetzt: Die Rolle kommt niemals aus dem Request-Body, sondern wird aus dem geprüften Token gelesen (siehe Kapitel 11).

ARCHITEKTUR · 09

9. Automatisierte Tests

Tests sind kleine Programme, die automatisch prüfen, ob sich der Code wie erwartet verhält, mit assert-Anweisungen („ich behaupte, das muss wahr sein — sonst Fehler“). Der wichtigste Test im Projekt beweist nicht nur „es funktioniert“, sondern den Worst Case:

python

```
def test_best_match_hidden_from_unauthorized_role():
    # Fast identischer Text wie Ticket 10 - waere OHNE Filter
    # garantiert der Top-Treffer.
    query = "Kuendigung eines Mitarbeiters in der Buchhaltung..."

    filtered = db.search(query, role="support", top_k=1)
    assert all(r["id"] != 10 for r in filtered)
```

Tests laufen bewusst immer mit dem kostenlosen HashingEmbedder statt mit Azure: So bleiben sie offline, kostenlos und reproduzierbar, unabhängig von einer Internetverbindung oder einem API-Key.

03

TEIL 3

Produktiv-Features

Passwort-Hashing · JWT-Login · Persistenz · Audit-Log · Multi-Tenancy

PRODUKTIV · 10

10. Passwort-Hashing

Passwörter werden niemals im Klartext gespeichert. Stattdessen läuft das Passwort durch eine **Hash-Funktion** (hier: bcrypt) — eine Einbahnstraße: Aus dem Passwort lässt sich ein Hash berechnen, aber aus dem Hash lässt sich das Passwort nicht zurückrechnen.

python

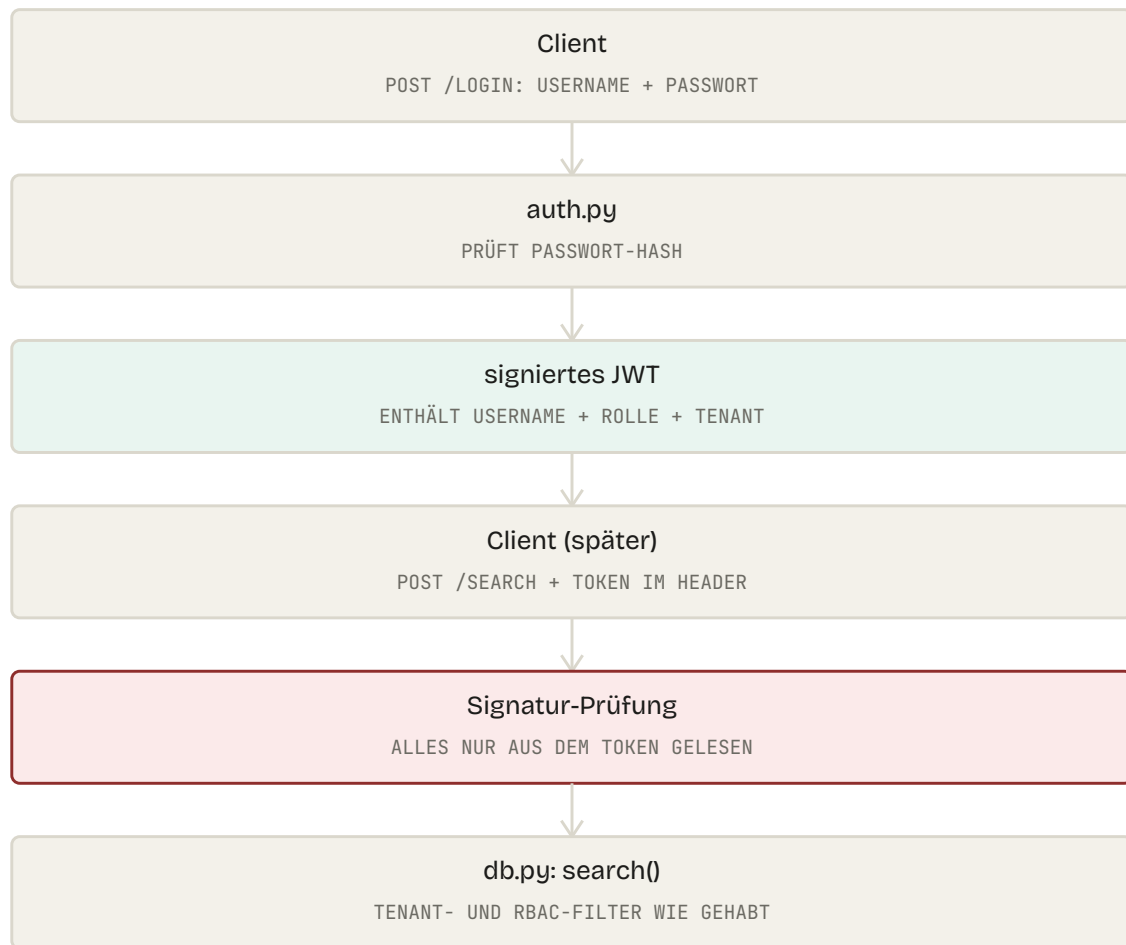
```
def hash_password(password: str) -> str:
    return bcrypt.hashpw(
        password.encode("utf-8"), bcrypt.gensalt()
    ).decode("utf-8")
```

Beim Login wird das eingegebene Passwort erneut gehasht und mit dem gespeicherten Hash verglichen. Würde die Datenbank jemals gestohlen, hätte ein Angreifer nur nutzlose Hashes, keine echten Passwörter.

PRODUKTIV · 11

11. JWT-Login

Nach erfolgreichem Login bekommt der Client ein **JWT** (JSON Web Token) — ein Stück Text mit Nutzdaten (username, role, tenant_id, Ablaufzeit) und einer digitalen Signatur, erzeugt mit einem geheimen Schlüssel, den nur der Server kennt. Bei jeder weiteren Anfrage schickt der Client dieses Token statt erneut Passwort einzugeben.



Der entscheidende Fortschritt: Vorher vertraute die API einem Client-Feld username und schlug die Rolle serverseitig nach — aber ein Client konnte sich trotzdem als anderer Nutzer ausgeben. Mit JWT ist auch das nicht mehr möglich: Username, Rolle und Mandant (`tenant_id`) stehen nur noch in einem signierten Token, das der Client sich nicht selbst ausstellen kann.

python

```

def get_current_user(credentials = Depends(_security)):
    try:
        return auth.decode_access_token(credentials.credentials)
    except jwt.PyJWTError:
        raise HTTPException(401, "Ungültiges oder abgelaufenes Token")
  
```

TIPP

Live getestet: Ein manipuliertes Token (letztes Zeichen geändert) und ein komplett fehlendes Token wurden beide mit HTTP 401 abgelehnt — die Signaturprüfung funktioniert wie vorgesehen.

PRODUKTIV · 12

12. SQLite-Persistenz

Ohne Persistenz leben alle Tickets nur im Arbeitsspeicher — ein Neustart löscht alles, und bei Azure-Embeddings würde jeder Neustart erneut kostenpflichtig jeden Text neu embedden. **SQLite** ist eine

Datenbank, die komplett in einer einzigen Datei lebt, kein separater Server nötig.

Wichtig für die Architektur: `db.py` (die Suchlogik) bleibt dabei komplett unverändert. Persistenz ist eine zusätzliche Schicht (`storage.py`), die beim Start Daten lädt — kein Umbau der Suche selbst.

python

```
def save_ticket(ticket_id, text, vector, allowed_roles, path=None):
    with _connect(path) as conn:
        conn.execute(
            "INSERT OR REPLACE INTO tickets VALUES (?, ?, ?, ?)",
            (ticket_id, text, json.dumps(vector.tolist()),
             json.dumps(allowed_roles)),
        )
```

WICHTIG

Echter Bug, den wir dabei live gefunden haben: Die Pfad-Parameter waren ursprünglich als `path: str = DB_PATH` definiert. Python wertet Default-Werte einmalig beim Laden der Datei aus, nicht bei jedem Aufruf — ein späteres Umsetzen von `storage.DB_PATH` (z.B. für Tests) hätte die Funktionen trotzdem den alten Pfad benutzen lassen. Fast wären dadurch Testdaten in die echte Datenbank geschrieben worden. Lösung: `path: str = None`, und der aktuelle Wert wird erst innerhalb der Funktion nachgeschlagen.

PRODUKTIV · 13

13. Authentication vs. Authorization

Zwei unterschiedliche Fragen, die leicht verwechselt werden. **Authentication** (Authentifizierung) beantwortet: *Wer bist du?* — das prüft das JWT bereits bei jeder Anfrage, siehe Kapitel 11. **Authorization** (Autorisierung) beantwortet eine zweite, davon unabhängige Frage: *Darfst du GENAU DAS hier tun?*

Ein gültig eingeloggter **support**-User ist authentifiziert — das Token ist echt, die Signatur stimmt. Trotzdem darf er nicht jeden Endpunkt benutzen. Für den Audit-Log-Endpunkt (Kapitel 14) reicht ein gültiges Token allein nicht aus; zusätzlich muss die Rolle passen.

python

```
def get_current_user(credentials = Depends(_security)):
    # Authentication: ist das Token echt und noch gueltig?
    return auth.decode_access_token(credentials.credentials)

def require_management(current_user = Depends(get_current_user)):
    # Authorization: baut auf Authentication auf, prueft zusaetzlich
    # das Recht fuer DIESEN Endpunkt.
    if current_user["role"] != "management":
        raise HTTPException(403, "Nur fuer management sichtbar")
    return current_user
```

TIPP

`require_management()` ruft `get_current_user()` auf, statt die Token-Prüfung zu duplizieren. Ein support-User, der versucht, `/audit-log` aufzurufen, bekommt dadurch nicht 401 (Login ungültig), sondern 403 (Login gültig, aber keine Berechtigung) — ein feiner, aber wichtiger Unterschied: der Client weiß dadurch, dass sein Token grundsätzlich funktioniert, nur eben nicht für diesen Endpunkt.

PRODUKTIV · 14

14. Audit-Log

Für Kunden, denen RBAC wichtig ist, reicht es oft nicht zu wissen, *dass* gefiltert wird — sie wollen im Nachhinein nachvollziehen können, *wer wann wonach gesucht hat*. Deshalb protokolliert `api.py` jede Suche: Username, Mandant, Anfrage, Trefferzahl, Zeitstempel.

Wichtiger Design-Punkt: Das Logging passiert in **`api.py`**, nicht in `db.py`. Die Suchmaschine selbst kennt gar keinen Username — nur eine Rolle und einen Tenant. Wer gesucht hat, ist reines Web-Schicht-Wissen.

python

```
@app.post("/search")
def search(request: SearchRequest, current_user = Depends(get_current_user)):
    results = db.search(request.query, role=current_user["role"],
                        tenant_id=current_user["tenant_id"],
                        top_k=request.top_k)
    storage.log_search(current_user["username"], current_user["tenant_id"],
                      request.query, len(results))
    return results
```

Der Log selbst ist wieder nur über **`require_management()`** erreichbar (Kapitel 13) — und zusätzlich nach Mandant gefiltert, sonst würde management eines Kunden sehen, wonach ein anderer Kunde gesucht hat (siehe Kapitel 15).

TIPP

Live getestet: Ein support-User führt eine Suche aus (wird protokolliert), versucht danach `/audit-log` aufzurufen — 403. Ein management-User ruft `/audit-log` auf — 200, mit dem Eintrag der Suche des support-Users.

PRODUKTIV · 15

15. Multi-Tenancy

Teilen sich mehrere Kunden (Mandanten) dieselbe Infrastruktur, reicht die Rolle allein nicht mehr aus. Zwei Mandanten können beide eine Rolle **management** haben, dürfen sich aber niemals gegenseitig sehen. Der Tenant-Filter ist deshalb eine zweite, **härtere und vorgelagerte** Schranke — er wird zuerst angewendet, noch vor dem Rollenfilter.

WICHTIG

`allowed_roles=["all"]` bedeutet dadurch „alle Rollen DIESES Mandanten“, nicht „alle Rollen weltweit“. Ein allgemeines Ticket von Mandant B bleibt für Mandant A trotzdem unsichtbar.

python

```
same_tenant = [e for e in self._entries if e["tenant_id"] == tenant_id]

allowed_entries = [
    e for e in same_tenant
    if "all" in e["allowed_roles"] or role in e["allowed_roles"]
]
```

Für den härtesten denkbaren Test haben wir zwei Mandanten mit **fast wortgleichen** sensiblen Tickets angelegt: Kanzlei Müller hat „Gehaltserhöhung... Budget... Quartal“, Steuerberatung Schmidt hat „Gehaltserhöhung... Quartal... Budget“ — fast dieselben Wörter, gleiche Rolle (management), aber unterschiedlicher Mandant.

python

```
query = "Gehaltserhoehung fuer das Team, mehr Budget pro Quartal"

# Tenant A (management) sucht - sieht NUR sein eigenes Ticket (8)
results_a = db.search(query, role="management", tenant_id="kanzlei-mueller")
assert all(r["id"] != 15 for r in results_a) # Ticket 15 gehoert Tenant B

# Tenant B (management) sucht dieselbe Anfrage - sieht NUR sein eigenes (15)
results_b = db.search(query, role="management", tenant_id="steuerberatung-schmidt")
assert all(r["id"] != 8 for r in results_b) # Ticket 8 gehoert Tenant A
```

Wie bei Rolle und Username gilt derselbe Grundsatz: **tenant_id** kommt ausschließlich aus dem geprüften JWT (Kapitel 11), nie aus einem Request-Feld — sonst könnte sich ein Nutzer einfach als anderer Mandant ausgeben.

04

TEIL 4

Projekt-Chronologie

Die konkreten Schritte, in der Reihenfolge, wie sie passiert sind

16. Die konkreten Schritte, in Reihenfolge

So ist das Projekt tatsächlich entstanden, Schritt für Schritt:

- **1. Grundkonzepte verstanden** — Vektor, Embedding, Cosine Similarity (von Hand durchgerechnet), Brute-Force, RBAC serverseitig mit Leck-Beispiel
- **2. Erste Version gebaut** — data.py (11 KMU-Support-Tickets, gemischt allgemein/sensibel), auth.py (simuliertes Dictionary), db.py (MiniVectorDB mit Platzhalter-Embedding), main.py (Demo), 5 Tests in test_rbac.py
- **3. Git-Repository angelegt** — lokal initialisiert, dann auf GitHub veröffentlicht (Arian710/mini-vector-rbac)
- **4. Business-Überlegungen** — Zielgruppe, Pricing, Markteintritt diskutiert, bevor weitergebaut wurde
- **5. Echte Azure-Embeddings vorbereitet** — embeddings.py mit austauschbarer Embedder-Abstraktion (HashingEmbedder + AzureOpenAIEmbedder), Dependency Injection in db.py
- **6. Web-API gebaut** — api.py mit FastAPI, Rolle zunächst aus Username serverseitig aufgelöst (noch ohne echtes Login)
- **7. Architektur erklärt und dokumentiert** — Diagramme, Interview-Fragenkatalog erarbeitet
- **8. GitHub Issues als User Stories angelegt** — #1 JWT-Login, #2 Persistenz, #3 Audit-Log (Backlog), #4 Multi-Tenancy (Backlog)
- **9. JWT-Login und SQLite-Persistenz umgesetzt** — storage.py, auth.py komplett neu (bcrypt + JWT), api.py mit /login und token-geschütztem /search, seed_data.py als Beispieldatensatz, test_auth.py
- **10. Gemeinsam live getestet** — Login als zwei Rollen, unterschiedliche Suchergebnisse, manipuliertes und fehlendes Token korrekt abgelehnt
- **11. Issues #1 und #2 automatisch geschlossen** — per „Closes #1, Closes #2“ in der Commit-Message
- **12. Audit-Log umgesetzt (#3)** — search_log-Tabelle in storage.py, /audit-log nur für management via require_management(), dabei einen latenten Bug gefunden: api.py/main.py übergaben MiniVectorDB() keinen Embedder, sodass Queries immer mit HashingEmbedder statt dem beim Seeden genutzten Embedder embedded worden wären
- **13. Multi-Tenancy umgesetzt (#4)** — tenant_id in Tickets, Usern, Tokens und Audit-Log; Tenant-Filter in db.py vor dem Rollenfilter; zweiter Demo-Mandant mit bewusst fast wortgleichem sensiblen Ticket zum härtesten Isolationstest
- **14. Gemeinsam live getestet** — zwei management-User in unterschiedlichen Mandanten, identische Anfrage, jeder sieht nur sein eigenes Ticket; Audit-Log ebenfalls mandantengetrennt bestätigt
- **15. Issues #3 und #4 automatisch geschlossen** — 18 Tests über vier Testdateien, README durchgängig aktualisiert

05

TEIL 5

Referenz

Interview-Fragenkatalog und Glossar zum Nachschlagen

REFERENZ · 17

17. Interview-Fragenkatalog

Erklär dein Projekt in 2 Sätzen.

Ich habe eine kleine Suchmaschine für interne Firmendokumente gebaut, die per KI-ähnlicher Bedeutungssuche funktioniert — aber so, dass ein Nutzer ein Dokument ohne Berechtigung nie zu sehen bekommt, selbst wenn es inhaltlich der beste Treffer wäre. Diese Zugriffsprüfung passiert zwingend im Backend, nicht in der Oberfläche.

Warum Cosine Similarity statt normaler Distanz?

Weil die Länge eines Embedding-Vektors technisch schwanken kann, ohne dass sich die Bedeutung ändert — Cosine schaut nur auf den Winkel/die Richtung.

Warum filterst du die Rolle vor der Berechnung, nicht danach?

Damit ein unerlaubtes Dokument strukturell nie Teil der Ergebnismenge werden kann, egal wie gut es inhaltlich passen würde.

Was wäre, wenn du den Filter im Frontend gebaut hättest?

Die sensiblen Daten wären trotzdem beim Client angekommen und über Entwicklertools oder einen direkten API-Aufruf sichtbar gewesen — „nicht angezeigt“ ist nicht dasselbe wie „nie da gewesen“.

Wie skaliert Brute-Force?

Linear mit der Datenmenge. Bei sehr großen Datenmengen würde man einen Index wie HNSW einsetzen (z.B. pgvector in Postgres), aber in unserer Größenordnung unnötig.

Warum zwei Embedder-Implementierungen?

Damit Tests offline, kostenlos und deterministisch laufen, während die echte Anwendung ein trainiertes Modell nutzt — ohne dass sich der Datenbank-Code ändern muss (Dependency Injection).

Wie stellst du sicher, dass sich ein Client nicht einfach eine andere Rolle zuschreibt?

Die Rolle wird nie aus dem Request gelesen, sondern ausschließlich aus einem signierten JWT, das der Server selbst ausgestellt hat. Ein Client kann ein Token nicht fälschen, ohne den geheimen Schlüssel zu kennen.

Was testen deine Tests genau?

Unter anderem den Worst Case: ein sensibles Dokument, das garantiert der beste inhaltliche Treffer wäre, darf eine nicht-berechtigte Rolle trotzdem nie zu sehen bekommen.

Was fehlt noch für echten Produktivbetrieb?

Audit-Logging (wer hat wann was gesucht), Multi-Tenancy für mehrere Kunden, HTTPS/Reverse-Proxy beim Deployment — alles bereits als GitHub Issues dokumentiert.

Warum SQLite und nicht gleich Postgres?

SQLite braucht keinen separaten Datenbankserver und ist für die Projektgröße völlig ausreichend. Die Persistenz-Schicht (storage.py) ist sauber von der Suchlogik getrennt, ein späterer Wechsel auf Postgres würde nur storage.py betreffen.

Was ist der Unterschied zwischen Authentication und Authorization?

Authentication prüft, ob jemand ist, wer er behauptet zu sein (gültiges Token). Authorization prüft zusätzlich, ob diese Person DIESEN konkreten Endpunkt benutzen darf. Ein gültig eingeloggter support-User ist authentifiziert, aber für /audit-log nicht autorisiert — 403, nicht 401.

Wie verhindert Multi-Tenancy, dass zwei Kunden mit gleicher Rolle sich sehen?

tenant_id ist ein zweiter Filter, der VOR dem Rollenfilter angewendet wird und aus dem geprüften JWT kommt, nie vom Client. Zwei Mandanten können beide die Rolle management haben — der Tenant-Filter verhindert trotzdem jede Überschneidung, weil er zuerst greift.

Warum liegt das Suche-Protokollieren in api.py und nicht in db.py?

db.py kennt gar keinen Username, nur Rolle und Tenant — das ist bewusst so, damit die Suchmaschine so wenig wie möglich über den Aufrufer wissen muss. Wer gesucht hat, ist Web-Schicht-Wissen und gehört deshalb in api.py.

REFERENZ · 18

18. Glossar

API

Programm, das über ein Netzwerk erreichbar ist und Anfragen (Requests) entgegennimmt und Antworten (Responses) zurückgibt.

Assert

Eine Zeile Code, die behauptet „das muss wahr sein“ — stimmt es nicht, bricht das Programm mit einer Fehlermeldung ab. Grundlage einfacher Tests.

Audit-Log

Ein dauerhaftes Protokoll darüber, wer wann was getan hat — hier: wer wann wonach gesucht und wie viele Treffer bekommen hat.

Authentication

Die Prüfung, ob jemand wirklich ist, wer er behauptet zu sein — bei uns: ein gültiges, unverfälschtes JWT.

Authorization

Die Prüfung, ob ein bereits authentifizierter Nutzer eine bestimmte Aktion/einen bestimmten Endpunkt benutzen darf.

bcrypt

Eine Hash-Funktion speziell für Passwörter — absichtlich langsam, um automatisiertes Ausprobieren zu erschweren.

Brute-Force-Suche

Eine Anfrage mit jedem einzelnen gespeicherten Eintrag vergleichen, statt einen Index zu benutzen.

Cosine Similarity

Ein Ähnlichkeitsmaß zwischen zwei Vektoren, das nur den Winkel zwischen ihnen misst, nicht ihre Länge.

Dependency Injection

Eine Komponente bekommt ihre Abhängigkeiten von außen übergeben, statt sie selbst zu beschaffen.

Embedding

Ein Vektor, der die Bedeutung von Text beschreibt, erzeugt von einem trainierten Modell.

FastAPI

Ein Python-Framework zum Bauen von Web-APIs.

Interface / Abstraktion

Ein Vertrag, der festlegt, was eine Komponente können muss, ohne vorzuschreiben, wie.

JSON

Ein einfaches, weit verbreitetes Text-Format für strukturierte Daten im Web.

JWT (JSON Web Token)

Ein signiertes Text-Token, das Nutzdaten (z.B. Username, Rolle, Tenant) enthält und vom Server ohne erneute Passwortabfrage geprüft werden kann.

Multi-Tenancy

Mehrere Kunden (Mandanten) teilen sich dieselbe Infrastruktur, ihre Daten bleiben aber strikt voneinander getrennt.

Pydantic

Eine Python-Bibliothek, die eingehende Daten automatisch gegen ein erwartetes Format prüft (Validierung).

RBAC

Role-Based Access Control — Zugriffsrechte werden über Rollen statt pro Person einzeln vergeben.

REST / Request / Response

Übliches Muster für Web-APIs: der Client schickt eine Anfrage (Request), der Server schickt eine Antwort (Response) zurück.

SQLite

Eine Datenbank, die komplett in einer einzigen Datei lebt, ohne separaten Datenbankserver.

Tenant

Ein Mandant: ein einzelner Kunde/eine einzelne Organisation innerhalb eines Multi-Tenancy-Systems.

Unit-Test

Ein kleines Programm, das automatisch prüft, ob sich ein Stück Code wie erwartet verhält.

Vektor

Eine Liste von Zahlen, die eine Position in einem (ggf. vieldimensionalen) Raum beschreibt.